

Coding Interview Grading Template

Use this template to evaluate a candidate consistently after a coding interview.
Adjust point values to fit the role, seniority, and interview format.

Candidate and Interview Details

? Candidate:
? Role/level:
? Interviewer:
? Date:
? Problem:
? Language:
? Interview format:
? Final recommendation: Strong hire / Hire / Lean hire / Lean no hire / No hire

Scoring Summary

Category	Points	Score	Notes
----------	--------	-------	-------

Problem understanding	15		
Algorithm and approach	20		
Code correctness	25		
Code quality and maintainability	15		
Testing and debugging	10		
Communication and collaboration	10		
Follow-up and complexity analysis	5		
Total	**100**		

Rubric

1. Problem Understanding (15 points)

- ? Clarifies requirements, inputs, outputs, and constraints.
- ? Identifies edge cases before or during implementation.
- ? Restates the problem accurately and confirms assumptions.

Score guidance:

- ? 13-15: Clear understanding with thoughtful clarifying questions.
- ? 9-12: Understands the main task but misses some constraints or edge cases.
- ? 5-8: Requires repeated prompting to identify core requirements.
- ? 0-4: Misunderstands the problem or solves a materially different task.

2. Algorithm and Approach (20 points)

- ? Chooses an appropriate data structure and algorithm.
- ? Explains tradeoffs and justifies the chosen approach.
- ? Adapts when an initial approach proves insufficient.

Score guidance:

- ? 17-20: Efficient, well-justified approach with clear tradeoff awareness.
- ? 13-16: Good approach with minor inefficiencies or gaps in explanation.
- ? 8-12: Partially correct approach that needs significant interviewer guidance.
- ? 0-7: Ineffective approach or no coherent strategy.

3. Code Correctness (25 points)

- ? Produces code that solves the stated problem.
- ? Handles typical cases and important edge cases.
- ? Avoids logical errors, off-by-one issues, and unsafe assumptions.

Score guidance:

- ? 22-25: Correct solution with robust edge-case handling.
- ? 17-21: Mostly correct with minor defects or omissions.
- ? 10-16: Partially working solution with meaningful correctness gaps.
- ? 0-9: Incomplete or incorrect solution.

4. Code Quality and Maintainability (15 points)

- ? Writes readable, idiomatic code for the chosen language.
- ? Uses clear names and simple control flow.
- ? Keeps functions focused and avoids unnecessary complexity.

Score guidance:

- ? 13-15: Clean, idiomatic, easy-to-maintain implementation.
- ? 9-12: Understandable code with some style or structure issues.
- ? 5-8: Hard-to-follow code, duplicated logic, or weak naming.
- ? 0-4: Code is difficult to reason about or maintain.

5. Testing and Debugging (10 points)

- ? Tests representative normal cases, boundary cases, and failure cases.
- ? Uses manual or automated checks effectively.
- ? Debugs issues methodically when results are unexpected.

Score guidance:

- ? 9-10: Strong test coverage and systematic debugging.
- ? 6-8: Tests common cases but misses some important boundaries.
- ? 3-5: Minimal testing or relies heavily on interviewer hints.
- ? 0-2: Does not test or cannot debug observed failures.

6. Communication and Collaboration (10 points)

- ? Thinks aloud and explains decisions clearly.
- ? Responds constructively to hints, questions, and feedback.
- ? Balances independent problem solving with collaboration.

Score guidance:

- ? 9-10: Clear, structured, and collaborative throughout.
- ? 6-8: Generally clear with occasional gaps in explanation.
- ? 3-5: Limited communication or difficulty incorporating feedback.
- ? 0-2: Communication blocks effective evaluation.

7. Follow-up and Complexity Analysis (5 points)

- ? States time and space complexity accurately.
- ? Discusses likely bottlenecks and reasonable optimizations.
- ? Handles follow-up questions about extensions or constraints.

Score guidance:

- ? 5: Accurate analysis and strong follow-up discussion.
- ? 3-4: Mostly accurate analysis with small omissions.
- ? 1-2: Incomplete or incorrect analysis.
- ? 0: No meaningful complexity discussion.

Interview Notes

Strengths

?
?
?

Concerns

?
?
?

Evidence and Examples

?
?
?

Follow-up Questions Asked

?
?
?

Candidate Questions

?
?
?

Recommendation Rationale

Summarize the decision using observable evidence from the interview. Note any role-specific considerations, signal strength, and areas where more evaluation would be useful.

Recommendation:

Rationale: